AIM, Explained From Scratch

How a differentially private synthetic-data mechanism chooses, measures and fits marginals

Technical study notes

2026-10-23

AIM is one of the best algorithms for releasing a synthetic copy of a table of personal records under differential privacy. This note explains it step by step, for a reader who knows basic probability and nothing about privacy or graphical models. It defines the task, the privacy guarantee and the two noisy building blocks AIM is made of. It then explains Private-PGM, the engine that turns a pile of noisy, mutually inconsistent statistics into one dataset, without the machinery its name suggests. The second half walks through AIM round by round on a toy dataset of a thousand people, with every number computed by a script, and proves that the whole procedure keeps its privacy promise. The results of the paper's experiments close the note.

Table of contents

1 The task: synthetic data that gets the marginals right **3**
- 1.1 Records, attributes and the full table . . . 4
- 1.2 Marginals . . . 4
- 1.3 The workload and the error . . . 5

2 Differential privacy, as much as AIM needs **5**
- 2.1 Neighbouring datasets and the promise . . . 5
- 2.2 Counting the cost in concentrated form . . . 6
- 2.3 The two building blocks . . . 7

3 Select, measure, generate **8**

4 Private-PGM, without graphical models **9**
- 4.1 One table that explains every measurement . . . 9
- 4.2 Why the table never has to be built . . . 10
- 4.3 From the table to records . . . 11

5 AIM, round by round **11**
- 5.1 The schedule . . . 11
- 5.2 Initialization . . . 12
- 5.3 The candidates . . . 12
- 5.4 Selection: the score . . . 13

5.5 Measurement and estimation . . . 16
5.6 Annealing: when a round teaches too little . . . 16
5.7 The run on the toy dataset . . . 17

6 Why AIM keeps its promise 19

7 What AIM says about its own error 20

8 How well it works 21

9 What the note proves and what it cites 22

10 Sources 22

An organization holds a table of records about people, one row each, with attributes such as age group, occupation and a medical condition. Researchers want to study the table, but it cannot leave the building. One answer is **synthetic data**, a new table of made-up rows. It is built to give nearly the same answers as the real one to the statistical questions that matter, while revealing almost nothing about any one real person. AIM, by McKenna, Mullins, Sheldon and Miklau (2022), is a way of building such a table. In the paper's experiments it beat every competitor on most settings.

The name stands for *adaptive and iterative mechanism*, and the three words say what it does. It is a mechanism, which in this field means a randomized algorithm whose privacy is analysed. It is iterative, because it builds its model of the data a piece at a time, each time choosing one small statistic to measure with noise. And it is adaptive, because each choice depends on what the earlier measurements taught it, and because it spends more privacy budget on a measurement when it finds that the earlier, cheaper ones taught too little.

Three questions organize the note. What does AIM promise, and how is that promise counted? How does a pile of noisy statistics become one dataset? And how does AIM decide what to measure next, and with how much noise? You need only basic probability to follow it: expectations, the normal distribution and a little calculus. Everything else is defined where it is first used. Table 1 lists the question each section answers, and Table 2 lists the symbols for reference. A script carries the running example, and every number the note quotes comes from it:

```
uv run python code/notes/aim.py
```

Table 1: The question each section answers.

Section	The question it answers
Section 1	What does synthetic data have to get right, and how is its error measured?
Section 2	What does the privacy promise say, and how are its costs counted?
Section 3	Why not measure every statistic directly, and what three problems that leaves?
Section 4	How does one dataset come out of many noisy, inconsistent statistics?

Section	The question it answers
Section 5	What does AIM do in each round, and why?
Section 6	Why does AIM keep its privacy promise?
Section 7	How does AIM know how wrong its answers are?
Section 8	How well does it work, and which of its ideas matter most?

Table 2: The symbols the note uses. Each is also explained where it first appears.

Symbol	Meaning
D, D'; $D \sim D'$	a dataset of n records, and a neighbouring one, which has one record more or one fewer
d	the number of attributes
r, s	sets of attributes
$\mu_r(D)$, m_r	the marginal of D over the attributes in r, a vector of counts, and its number of cells
$\mathcal{Q}$, c_s	the workload, a list of attribute sets whose marginals the synthetic data should get right, with their weights
$\mathcal{Q}_+$	the downward closure of the workload, the marginals AIM may measure
w_r	the weight AIM gives a candidate marginal r
$\hat{p}$, $\hat{p}_t$	the model's table of counts, and the model after round t
$\hat{D}$	the synthetic dataset
ε, δ	the parameters of a differential-privacy guarantee
ρ, ρ_{used}	the privacy budget in zero-concentrated form, and the part of it spent so far
σ_t, ε_t	the noise level and the selection budget of round t; σ_0 is the starting noise level
T	the number of rounds AIM plans for
$u(D, r)$, Δu	the score AIM gives candidate r, and the most one record can change it
$D_\alpha(P\|Q)$	the Rényi divergence of order α between two distributions
θ	the parameters of the model

1 The task: synthetic data that gets the marginals right

Before any privacy enters, we need to say what a good synthetic dataset is. This section fixes the shape of the data, defines the statistics that AIM works with, called marginals, and states the error that the paper measures. The running example of the note is introduced here too.

1.1 Records, attributes and the full table

A dataset D is a list of n records, and each record gives a value for each of d attributes. AIM requires every attribute to be **discrete**, which means that it takes one of a fixed, finite list of values. An attribute such as income has to be cut into bands first.

The running example is a toy dataset of $n = 1{,}000$ people with four attributes. *Smoker* has the values no and yes, and *exercise* the values rarely and often. *Age* has three bands, under 40, 40 to 64, and 65 and over. *Condition* records whether the person has a certain medical condition. The script draws the dataset from a fixed population in which smoking, exercise and the condition all depend on age, and the condition also depends on smoking, so there is structure for AIM to find.

A record is one combination of values, and the number of possible combinations is the product of the attributes' sizes. For the toy dataset that is $2 \times 3 \times 2 \times 2 = 24$ combinations. The **full table** of a dataset counts how many records fall on each combination. It holds everything there is to know about the dataset, apart from the order of the rows, and the whole of this note can be read as the question of how to release a noisy version of it. For real data it is far too big to release or even to store, because the product of fifteen attribute sizes runs into the billions, and most of its cells hold zero or one person. That is why AIM works with marginals instead.

1.2 Marginals

A marginal looks at a few attributes at a time and ignores the rest.

Definition 1.1 (Marginal)**.** Let r be a set of attributes. The **marginal** of D over r, written $\mu_r(D)$, is the vector that gives, for every combination of values of the attributes in r, the number of records of D with that combination. Its number of entries, or **cells**, is written m_r. A marginal over one attribute is a **one-way** marginal, over two attributes a **two-way** marginal, and so on.

So a marginal is a contingency table, flattened into a vector. Table 3 shows the toy dataset's marginal over smoker and condition, which has $m_r = 4$ cells.

Table 3: The marginal of the toy dataset over smoker and condition: how many of its 1,000 people fall in each of the four cells. Its one-way marginals are the row and column sums.

smoker	condition	no	yes
no		627	98
yes		181	94

Two facts about marginals drive everything that follows. First, a marginal over a set r determines the marginals over every subset of r, by adding up cells: the row sums of Table 3 are the one-way marginal of smoker. Second, every marginal is a sum over the full table. Summing the full table over the attributes outside r gives $\mu_r(D)$, and this is a linear operation, which is what lets Section 4 fit a table to noisy marginals.

1.3 The workload and the error

Whoever asks for the synthetic data says which marginals they care about. The **workload** $\mathcal{Q}$ is a list of attribute sets. Each set s in it comes with a weight, written c_s, that says how much that marginal matters. In the toy example the workload is all six two-way marginals, with equal weights. In the paper's experiments it is typically every three-way marginal of a dataset, several hundred of them.

The synthetic dataset $\hat{D}$ is a dataset of the same shape as D, so its marginals are computed in the same way. Its quality is the weighted distance between its workload marginals and the true ones. The paper measures distance by the L_1 norm, the sum of the absolute differences of the cells, and divides by the number of marginals and by n:

$$\text{Error}(D, \hat{D}) = \frac{1}{|\mathcal{Q}|\, n} \sum_{s \in \mathcal{Q}} c_s \left\| \mu_s(D) - \mu_s(\hat{D}) \right\|_1. \tag{1}$$

Dividing by n turns counts into shares of the population, so an error of 0.05 means that a typical workload marginal is off by five percent of the records in total, spread over its cells. The toy run in Section 5 starts at about 0.14 and ends at about 0.05.

2 Differential privacy, as much as AIM needs

AIM promises that the synthetic data reveal almost nothing about any one person, a promise called differential privacy (DP). This section says exactly what that means, introduces the way of counting privacy costs that AIM uses, and presents the only two noisy operations AIM ever performs on the data. Everything AIM does to the data is one of these two operations, so once their costs are known, the privacy of the whole algorithm is an exercise in adding them up.

2.1 Neighbouring datasets and the promise

The promise is stated by comparing two datasets that differ in one person.

Definition 2.1 (Neighbouring datasets)**.** Two datasets D and D' are **neighbouring**, written $D \sim D'$, if one can be obtained from the other by adding or removing a single record.

Some papers instead call datasets neighbouring when one record is replaced by another. AIM uses the add-or-remove version, and the choice matters for the noise levels below, because adding a record changes a marginal in one cell, while replacing one changes two cells.

A **mechanism** M is a randomized algorithm that takes a dataset and returns an output, here a synthetic dataset. Because it is random, its output on D has a probability distribution, and the promise is that this distribution barely changes when D is replaced by a neighbour.

Definition 2.2 (Differential privacy)**.** Let $\varepsilon \geq 0$ and $\delta \in [0, 1]$. A mechanism M is (ε, δ)-**differentially private**, or (ε, δ)-DP, if for every pair of neighbouring datasets $D \sim D'$ and every set S of possible outputs,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{2}$$

In words, no outcome becomes much more likely because one person's record is in the data. For small ε the factor e^{ε} is about $1 + \varepsilon$, so $\varepsilon = 1$ allows an outcome to become at most about 2.7 times more likely, and δ is a small probability of exceptions, typically far below one in a million. Since the definition covers both orders of the pair, it also bounds the other direction. An observer who sees the output therefore cannot tell with any confidence whether a given person's record was used, whatever else they know.

2.2 Counting the cost in concentrated form

AIM performs many noisy operations, and the privacy of the whole must be computed from the privacy of the parts. Adding up (ε, δ) pairs directly gives weak totals, so AIM counts in a different currency, called zero-concentrated differential privacy, in which costs simply add and convert to an (ε, δ) pair at the end. Its definition uses a way of measuring the distance between two distributions.

Definition 2.3 (Rényi divergence)**.** Let P and Q be two distributions on the same outputs, and write p and q for their densities, or their mass functions if the outputs are discrete. Let the order α be a number greater than 1. Their **Rényi divergence of order** α is

$$D_{\alpha}(P \| Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_{Y \sim Q} \left[\left(\frac{p(Y)}{q(Y)} \right)^{\alpha} \right]. \tag{3}$$

The ratio $p(y)/q(y)$ says how much more likely an output is under P than under Q, and the divergence is an average of a power of that ratio, on a logarithmic scale. It is zero when the distributions are equal and grows as they separate, and a higher order α weights the outputs where the ratio is largest more heavily.

Definition 2.4 (Zero-concentrated differential privacy)**.** Let $\rho \geq 0$. A mechanism M is ρ-**zero-concentrated differentially private**, or ρ-zCDP, if for every pair of neighbouring datasets $D \sim D'$ and every order $\alpha > 1$,

$$D_{\alpha}(M(D) \,\|\, M(D')) \leq \rho\, \alpha. \tag{4}$$

The number ρ is the **privacy budget** in this currency. The three facts AIM needs about it are standard, and the note cites their proofs.

Proposition 2.1 (Three properties of zCDP)**.**

(a) Composition. *If M_1 is ρ_1-zCDP and M_2 is ρ_2-zCDP, where M_2 may be chosen after seeing the output of M_1, then releasing both outputs is $(\rho_1 + \rho_2)$-zCDP.*

(b) Post-processing. *Any computation on the output of a ρ-zCDP mechanism, which does not look at the data again, is still ρ-zCDP.*

(c) Conversion. *A ρ-zCDP mechanism is (ε, δ)-DP for every $\varepsilon \geq 0$ with*

$$\delta = \min_{\alpha > 1} \frac{\exp\big((\alpha - 1)(\alpha\rho - \varepsilon)\big)}{\alpha - 1} \left(1 - \frac{1}{\alpha}\right)^{\alpha}. \tag{5}$$

Bun and Steinke (2016) prove parts (a) and (b), and Canonne, Kamath and Steinke (2020) prove part (c). All three proofs work with the Rényi divergence directly and need nothing beyond its definition. Part (a) is the whole reason for the currency: the costs of AIM's rounds add, even though each round's choices depend on the previous rounds' outputs. Part (b) is what makes most of AIM free. Fitting a model to released noisy marginals, deciding what to measure next from that model, and rounding the model to a dataset all look only at released values, so they cost nothing. Part (c) is how a budget is set. The user picks ε and δ, and the script finds the largest ρ whose conversion gives them. For $\varepsilon = 1$ and $\delta = 10^{-6}$, the values the toy run uses, the budget is $\rho = 0.0244$.

2.3 The two building blocks

The first building block releases a vector of counts with Gaussian noise. What it costs depends on how much one person can change the vector.

Definition 2.5 (Sensitivity)**.** Let f map datasets to vectors. Its L_2 **sensitivity** is the largest Euclidean distance between its values on two neighbouring datasets,

$$\Delta_2 f = \max_{D \sim D'} \big\| f(D) - f(D') \big\|_2. \tag{6}$$

A marginal has the smallest sensitivity a count can have.

Lemma 2.1 (A marginal has sensitivity one)**.** *For every attribute set r, the marginal μ_r has L_2 sensitivity* 1*.*

Proof. Adding one record to D adds one to exactly one cell of $\mu_r(D)$, the cell of that record's values on the attributes in r, and leaves every other cell unchanged. Removing a record subtracts one from one cell. In both cases the difference vector has a single entry of ± 1, whose Euclidean length is 1. □

Proposition 2.2 (The Gaussian mechanism)**.** *Let f have L_2 sensitivity $\Delta_2 f$, and let $\sigma > 0$. The mechanism that releases $f(D) + Z$, where Z is a vector of independent $\mathcal{N}(0, \sigma^2)$ noise, one entry per coordinate, is ρ-zCDP with*

$$\rho = \frac{(\Delta_2 f)^2}{2\sigma^2}. \tag{7}$$

Bun and Steinke (2016) prove this for their Gaussian mechanism proposition. The proof computes the Rényi divergence between two normal distributions with the same variance, which is α times the squared distance of their means over $2\sigma^2$, and bounds the distance by the sensitivity. For a marginal, by Lemma 2.1, a measurement with noise level σ costs $1/(2\sigma^2)$, whatever the marginal's size. That independence from the size is deceptive. A marginal with many cells spreads the same n records thinly, so each cell's count is small compared with the noise, which is why Section 5 will prefer smaller marginals when the budget is tight.

The second building block chooses one option from a list, favouring options with a high score, without revealing the scores.

Proposition 2.3 (The exponential mechanism)**.** *Let $\mathcal{R}$ be a finite set of candidates, and let $u(D, r)$ be a score for each candidate r, whose* ***sensitivity*** *Δu is the most that adding or removing one record can change any candidate's score. The mechanism that chooses candidate r with probability proportional to $\exp(\varepsilon\, u(D, r)/(2\Delta u))$ is ε-DP, and it is ρ-zCDP with*

$$\rho = \frac{\varepsilon^2}{8}. \tag{8}$$

The ε-DP claim is the original theorem of McSherry and Talwar (2007). The zCDP cost is proved by Cesar and Rogers (2021), from the fact that the mechanism's privacy loss is not only bounded by ε but confined to an interval of length ε. The exponent is what makes the mechanism useful. A candidate whose score is higher by $2\Delta u/\varepsilon$ is e times more likely to be chosen. So with a large ε the mechanism almost always picks the best candidate, and with a tiny ε it picks nearly at random. Section 5 shows both regimes on the toy dataset.

3 Select, measure, generate

Why does AIM not simply measure every marginal in the workload with Gaussian noise and release the results? This section works out what that costs, and finds the three problems that any better method has to solve. AIM belongs to a family of methods, which the paper calls *select-measure-generate*, that solve them in a common pattern.

Suppose the workload has k marginals and each is measured once with the same noise level σ. By Proposition 2.2 and Proposition 2.1 (a), the total cost is $k/(2\sigma^2)$, so spending the whole budget gives $\sigma = \sqrt{k/(2\rho)}$. On the toy workload, with $k = 6$ and $\rho = 0.0244$, that is $\sigma = 11.1$, which is small next to counts in the hundreds. On a workload of all three-way marginals of fifteen attributes, with $k = 455$, it is $\sigma = 96.6$. A three-way marginal of a dataset with fifty thousand records might have a hundred cells averaging five hundred records each, so the noise would be a fifth of a typical count, and far more than most of the smaller cells. The budget, in short, does not stretch to every marginal at once. That is the first problem, **selection**: measure only the marginals that matter, and let the rest be inferred from them.

The second problem appears even on the toy workload. The six noisy marginals are not a dataset. Two of them that share an attribute will disagree about its one-way marginal, because each carries its own noise. Some cells will be negative, which no count can be. And a researcher who wants a marginal that was never measured, such as a three-way one, gets no answer at all. This is the

problem of **generation**: turn the noisy measurements into one dataset whose marginals are all consistent, non-negative, and close to the measurements. Section 4 is about this problem alone.

The third problem is the **budget split**. A measurement with little noise costs a great deal, and a measurement with much noise may be worthless. Spreading the budget evenly over the measurements, as the calculation above did, ignores the fact that some marginals are far from what the model already knows and others are not.

The select-measure-generate family answers the three problems in a loop. Select a marginal, measure it with noise, update a model of the data to agree with every measurement so far, and repeat, then generate a dataset from the model at the end. The ancestor of AIM in this family is a method called the multiplicative weights exponential mechanism (MWEM), whose selection step picks the marginal on which the current model is most wrong, by the exponential mechanism, and whose model is updated by a simple multiplicative rule. AIM keeps the loop and changes every step in it, as Section 5 explains. But first the generation step, which both share.

4 Private-PGM, without graphical models

The name Private-PGM promises probabilistic graphical models, and the paper that introduced it (McKenna, Sheldon and Miklau, 2019) uses their full machinery. This section explains what the method computes without that machinery, because what it computes is simple. It then says, in words, why the machinery is needed to compute it at scale, and what the one number that AIM takes from it, the size of the model, means.

4.1 One table that explains every measurement

Suppose that a sequence of marginals has been measured. The i-th measurement is a noisy vector $\tilde{y}_i$, equal to $\mu_{r_i}(D)$ plus Gaussian noise of level σ_i in each cell. We want one dataset that agrees with all of them. Private-PGM's answer is to look for a full table p, a vector of non-negative numbers over all combinations of values whose entries sum to the number of records, and to ask that its marginals be as close to the measurements as possible. A table's marginal $\mu_r(p)$ is computed exactly as a dataset's, by summing over the attributes outside r. The closeness is measured by squared error, weighted so that noisier measurements count for less:

$$\hat{p} = \arg\min_{p} \sum_{i} \frac{1}{\sigma_i} \left\| \mu_{r_i}(p) - \tilde{y}_i \right\|_2^2. \tag{9}$$

Squared error is the natural choice because the noise is Gaussian. If the weights were $1/\sigma_i^2$, the minimizer would be exactly the maximum-likelihood table, the one under which the observed measurements were most probable. The paper uses $1/\sigma_i$, which still weights the precise measurements more and worked better in practice.

Two problems from Section 3 vanish at once. The marginals of one table cannot disagree with each other, because they are all sums of the same numbers, and they cannot be negative, because the table is not. The third benefit is quieter. A table answers every marginal, including the ones never measured, by combining what the measured ones imply. Table 4 shows how. After the toy run has measured only the four one-way marginals, the model's marginal over smoker and

condition keeps the measured row and column totals and spreads them as if the two attributes were independent, because nothing measured says otherwise.

Table 4: The true marginal over smoker and condition, and what the model fitted to the noisy one-way marginals alone gives for it. The model keeps the row and column sums, within the noise, but it spreads each row in the same proportions, because nothing it has measured says that smoking and the condition go together.

smoker	true: no condition	true: condition	model: no condition	model: condition
no	627	98	584.8	140.4
yes	181	94	236.6	56.8

The number of records n needs one remark. Under add-or-remove neighbours, n is itself private, since it changes between neighbouring datasets, so the table's total cannot be set to n. Each noisy one-way marginal sums to a noisy copy of n, and the script uses the average of these sums, a post-processing step that costs nothing. The toy run estimates 1,019 records.

4.2 Why the table never has to be built

The difficulty is size. On the toy domain, p has 24 entries, and the script simply searches over them. On a real domain with billions of combinations, no computer can hold p, let alone optimize it. The observation that rescues Equation 9 is that the objective depends on p only through its marginals over the measured sets $r_1, r_2, \ldots$, and that forces the shape of the solution.

Theorem 4.1 (The solution is a product of small tables)**.** *Write x_{r_i} for the combination of values that a record x takes on the attributes in r_i. Among the minimizers of Equation 9 there is a table of the form*

$$p_\theta(x) \propto \exp\Big(\sum_i \theta_i[x_{r_i}]\Big), \tag{10}$$

where each θ_i is a table of numbers with one entry per cell of the i-th measured marginal.

McKenna, Sheldon and Miklau (2019) prove this by a convex duality argument. The objective of Equation 9 depends on the table only through linear functions of it, the marginals, and the maximum-entropy table with given marginals has this exponential form. The practical content is that the model has as many parameters as the measurements have cells, however large the domain. Fitting it means adjusting the parameter tables θ_i until the model's marginals match the measurements, which is what the script's optimizer does, and what Private-PGM does with a more careful algorithm.

The one-way case shows the form at work. With only one-way marginals measured, the model is proportional to $\exp(\theta_1[x_1])\exp(\theta_2[x_2])\cdots$, a product of one factor per attribute, which is the definition of independent attributes. That is why Table 4 shows an independence table. After the marginal over smoker and condition is measured, the model gains a factor that depends on both attributes, and only that pair becomes dependent. In general, the model carries exactly

the dependencies that were measured and no others, which is the most cautious reading of the measurements.

What the machinery of graphical models adds is a way to compute the marginals of p_θ without ever writing out p_θ. The sum over all combinations of a product of factors can be reorganized attribute by attribute, summing out one attribute at a time, and the cost is set by the largest intermediate table that this process has to form. That largest table is governed by how the measured attribute sets overlap. Sets that form chains and small clusters keep it small, while many sets that overlap in a tangled way make it enormous. The paper calls the memory the computation needs the **junction tree size**, written JT-SIZE$(r_1, \dots, r_t)$ for the measured sets, in megabytes, and it is the one thing from the graphical-model world that AIM reads. Section 5 shows how AIM keeps it below a limit.

4.3 From the table to records

The fitted table $\hat{p}$ has fractional entries, and a dataset needs whole records. Private-PGM produces the synthetic dataset $\hat{D}$ by rounding. Each cell's count is rounded down, and the records lost to rounding are given back to the cells with the largest fractional parts, with a random tie-break in the original. Sampling records from $\hat{p}$ would also work, but the rounding adds less error. By Proposition 2.1 (b) this step is free. In the toy run, the model's workload error is 0.0529 and the rounded dataset's is 0.0537.

5 AIM, round by round

This is the heart of the note. AIM takes a dataset D, a workload $\mathcal{Q}$ with weights c_s, and a budget ρ, and returns a synthetic dataset. It has three settings, which the paper fixes once and for all. It plans for $T = 16d$ rounds, where d is the number of attributes. It gives nine tenths of each round's budget to the measurement and one tenth to the selection. And it caps the model's junction tree size at 80 megabytes. The sections below follow the algorithm in order, and Table 7 at the end shows every round on the toy dataset.

5.1 The schedule

Each round t selects a marginal with the exponential mechanism at a budget ε_t and measures it with Gaussian noise at a level σ_t. By Proposition 2.3, Proposition 2.2 and Lemma 2.1, the round costs

$$\rho_t = \frac{\varepsilon_t^2}{8} + \frac{1}{2\sigma_t^2}. \tag{11}$$

AIM chooses the starting values so that T rounds at those values would spend the budget in the planned split. Nine tenths of ρ over T measurements means $T/(2\sigma_0^2) = 0.9\rho$, and one tenth over T selections means $T\varepsilon_1^2/8 = 0.1\rho$. Solving each for its unknown gives

$$\sigma_0 = \sqrt{\frac{T}{2 \cdot 0.9\,\rho}}, \qquad \varepsilon_1 = \sqrt{\frac{8 \cdot 0.1\,\rho}{T}}. \tag{12}$$

These are deliberately cautious. With $T = 16d$ the starting noise is large, and the starting selection budget is tiny. The schedule is only a starting point, because the annealing rule below raises the per-round budget whenever a round turns out to teach too little, and AIM stops when the budget is gone rather than after T rounds. Table 5 gives the schedule for the toy dataset, where $d = 4$.

Table 5: AIM's budget schedule on the toy dataset, with the paper's defaults. Each annealing doubles ε and halves σ, so it multiplies the cost of a round by four.

Quantity	Value
budget ρ, from $(\varepsilon, \delta) = (1, 10^{-6})$	0.0244
rounds planned, $T = 16d$	64
starting noise $\sigma_0 = \sqrt{T/(2 \cdot 0.9\rho)}$	38.2
starting selection budget $\varepsilon_1 = \sqrt{8 \cdot 0.1\rho/T}$	0.0174
cost of the initialization, $d/(2\sigma_0^2)$	0.0014
cost of one round at the start, $\varepsilon_1^2/8 + 1/(2\sigma_0^2)$	0.0004
cost of a round after one annealing	0.0015
cost of a round after two annealings	0.0061

5.2 Initialization

Before the loop, AIM measures every one-way marginal that appears in some workload marginal, each with noise level σ_0, and fits the model to them. The cost is at most $d/(2\sigma_0^2)$, which by Equation 12 equals $0.9\rho\, d/T$, a sixteenth of the measurement budget when $T = 16d$. The reason is economy. The paper observed that MWEM's early rounds go to one-way marginals anyway, each paid for with a selection as well as a measurement, so measuring them outright saves the selections. The result is the independence model of Table 4, which already has every attribute's distribution about right.

5.3 The candidates

Which marginals may AIM measure? Not only the workload's, but every marginal over a subset of a workload set. The paper calls this collection the **downward closure** of the workload,

$$\mathcal{Q}_+ = \{\, r : r \subseteq s \text{ for some } s \in \mathcal{Q} \,\}. \tag{13}$$

A large marginal spreads the records over many cells, so each cell's count is small and the noise swamps it. Its sub-marginals have fewer cells and bigger counts, and by Theorem 4.1 the model combines them into an estimate of the large one. When the budget is small, AIM therefore often does better to measure pieces of a workload marginal than the marginal itself. On the

toy workload of six two-way marginals, the closure adds the four one-way marginals, for ten candidates in all.

Each candidate gets a weight that says how much of the workload it serves. The weight adds up, over the workload marginals, the number of attributes the candidate shares with each, times that marginal's weight:

$$w_r = \sum_{s \in \mathcal{Q}} c_s \, |r \cap s|. \tag{14}$$

A two-way candidate on the toy workload shares both attributes with itself and one attribute with four other workload marginals, so its weight is 6, while a one-way candidate shares its attribute with three workload marginals, for a weight of 3.

Finally, in each round AIM drops the candidates that would make the model too big. A candidate r stays in the round's candidate set only if the junction tree size of the measured sets plus r is at most $(\rho_{\text{used}}/\rho)$ times the 80 megabyte cap. The allowance grows with the fraction of the budget spent, so early rounds, which have measured little, are kept to small models, and the full cap is available only near the end. On the toy domain every model is tiny, so the script does not need this filter.

5.4 Selection: the score

Here is AIM's first real idea. MWEM scores a candidate by how wrong the current model is on it, the L_1 distance $\|\mu_r(D) - \mu_r(\hat{p}_{t-1})\|_1$ between the true marginal and the model's. That rewards large marginals, which have large errors simply because they have many cells, even when measuring them would replace the model's error by a larger error from the noise. AIM subtracts what the noise would cost. The noise cost is the expected L_1 norm of the noise vector, which needs one lemma.

Lemma 5.1 (Expected size of Gaussian noise)**.** *If $Z \sim \mathcal{N}(0, \sigma^2)$, then $\mathbb{E}|Z| = \sigma\sqrt{2/\pi}$. Hence a vector of m independent such entries has expected L_1 norm $\sigma m\sqrt{2/\pi}$.*

Proof. By symmetry of the density about zero, the expectation of $|Z|$ is twice the integral over the positive half-line:

$$\mathbb{E}|Z| = 2\int_0^\infty \frac{z}{\sigma\sqrt{2\pi}} \exp\left(-\frac{z^2}{2\sigma^2}\right) \mathrm{d}z. \tag{15}$$

The integrand is, up to the constant in front, the derivative of $-\sigma^2 \exp(-z^2/(2\sigma^2))$, which runs from $-\sigma^2$ at zero to 0 at infinity. So the integral equals $\sigma^2/(\sigma\sqrt{2\pi})$, and doubling it gives $2\sigma/\sqrt{2\pi} = \sigma\sqrt{2/\pi}$. The L_1 norm of a vector is the sum of the absolute values of its entries, and expectation is linear, so m entries give m times the single-entry value. □

The constant $\sqrt{2/\pi}$ is about 0.798. AIM's score for candidate r in round t is then

$$u(D, r) = w_r\Big(\big\|\mu_r(D) - \mu_r(\hat{p}_{t-1})\big\|_1 - \sqrt{2/\pi}\,\sigma_t\, m_r\Big). \tag{16}$$

Read it as a balance in units of records. The first term is how far the model is from the truth on r, and the second is how far the noisy measurement would be from the truth, in expectation. A candidate earns a positive score only when measuring it is expected to leave the model closer to the truth than it is now, and the weight scales the balance by how much of the workload the candidate serves. The script's version is short:

```python
def score(
    data: Array,
    model: Array,
    attrs: Attrs,
    weight: float,
    sigma: float,
) -> float:
    """AIM's quality score of a candidate marginal: how far the model is from the data
    on it, minus the L1 error that measuring it with noise of standard deviation sigma
    would add in expectation, times the weight."""
    model_error = float(np.abs(marginal(data, attrs) - marginal(model, attrs)).sum())
    penalty = SQRT_2_OVER_PI * sigma * cells(attrs)
    return weight * (model_error - penalty)
```

The exponential mechanism needs the score's sensitivity, and the weight sets it.

Proposition 5.1 (Sensitivity of the score)**.** *Adding or removing one record changes $u(D, r)$ by at most w_r. So, over a candidate set $\mathcal{R}$, the sensitivity is $\Delta u = \max_{r \in \mathcal{R}} w_r$.*

Proof. The model $\hat{p}_{t-1}$ and the noise level are fixed when the score is computed, so only $\mu_r(D)$ changes between neighbouring datasets, and by the argument of Lemma 2.1 it changes in a single cell by 1. The L_1 distance to the fixed vector $\mu_r(\hat{p}_{t-1})$ therefore changes by at most 1, by the triangle inequality applied to that one cell. Multiplying by w_r gives the claim, and the sensitivity over the set is the largest of these. □

AIM then chooses r_t with probability proportional to $\exp(\varepsilon_t u(D, r)/(2\Delta u))$, as Proposition 2.3 prescribes. Table 6 and Figure 1 show the first round on the toy dataset. The model is the independence model, so the candidates with real dependence to find, such as smoker with condition and age with exercise, have the largest errors, and two of them clear their noise penalty by a wide margin.

Table 6: The first round's selection on the toy dataset. The model error is the L_1 distance between the true marginal and the model's, the noise penalty is $\sqrt{2/\pi}\,\sigma_1$ times the marginal's cells, the score is the weight times their difference, and the last column is the exponential mechanism's probability of choosing that candidate at the first round's ε_1.

Candidate	w_r	Model error on it	Noise penalty	Score	Chance of being chosen
smoker	3	19	61	-127	9%
age	3	104	91	38	11%

Candidate	w_r	Model error on it	Noise penalty	Score	Chance of being chosen
exercise	3	38	61	-70	9%
condition	3	19	61	-127	9%
smoker × age	6	114	183	-416	6%
smoker × exercise	6	38	122	-506	5%
smoker × condition	6	177	122	333	17%
age × exercise	6	249	183	396	18%
age × condition	6	181	183	-9	10%
exercise × condition	6	88	122	-203	8%

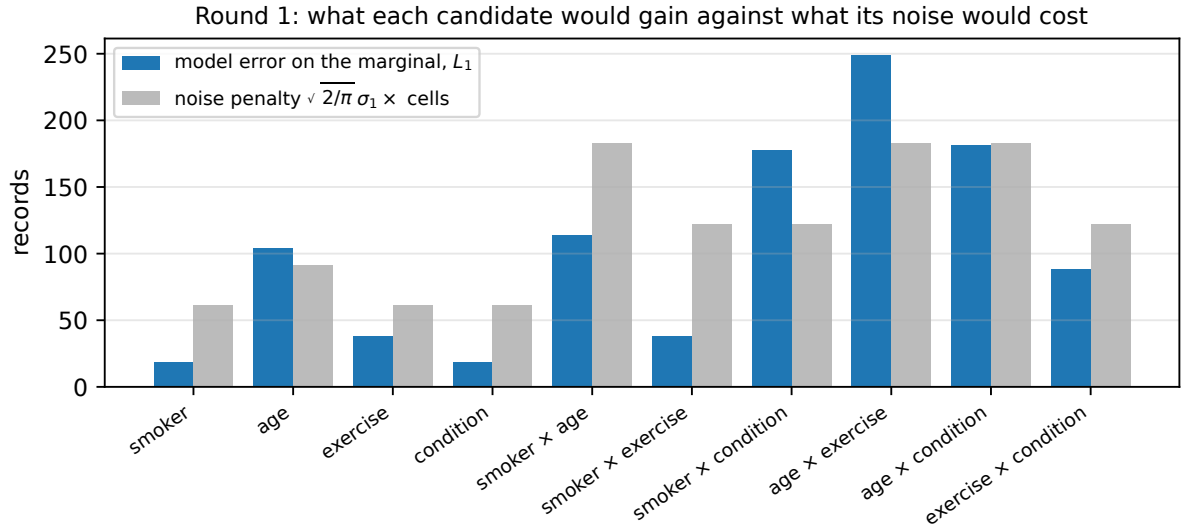


Figure 1: The first round's balance for each candidate: the model's error on the marginal against the L_1 error that measuring it would add in expectation. Only candidates whose blue bar is taller than their grey bar get a positive score.

The last column of Table 6 is the sobering part. At the starting budget $\varepsilon_1 = 0.0174$, the best candidate's score of 396 translates into an exponent of $0.0174 \times 396/12 \approx 0.57$, so it is only about $e^{0.57} \approx 1.8$ times as likely as an average candidate. The selection is nearly random, and in the toy run round 1 chose the one-way marginal of smoker, which the model already knew. This is not a flaw in the score. It is what the cautious schedule of Equation 12 buys, and the annealing rule exists to notice it and raise the stakes.

5.5 Measurement and estimation

The chosen marginal is measured with Gaussian noise at level σ_t, by Proposition 2.2, and appended to the list of measurements. Then the model is refitted to the whole list by Equation 9, which by Proposition 2.1 (b) is free. Refitting to everything, rather than adjusting the model for the new measurement alone, is what keeps the model consistent with every earlier measurement, weighted by its precision.

5.6 Annealing: when a round teaches too little

Here is AIM's second idea, which makes the number of rounds and the per-round budget automatic. After refitting, AIM compares the model's marginal over r_t before and after the measurement. If the measurement moved it by no more than the noise it carried, that is, if

$$\left\|\mu_{r_t}(\hat{p}_t) - \mu_{r_t}(\hat{p}_{t-1})\right\|_1 \le \sqrt{2/\pi}\,\sigma_t\, m_{r_t}, \tag{17}$$

then the round taught too little, and AIM doubles ε_{t+1} and halves σ_{t+1}. By Equation 11 both changes multiply the cost of a round by four. The condition can hold for two reasons, and the response fixes both. Either no candidate was worth measuring at this noise level, in which case halving σ makes more candidates' scores positive, or a good candidate existed and the near-random selection missed it, in which case doubling ε makes the selection sharper. The comparison looks only at two fitted models, so it is free.

A second rule ends the loop cleanly. After the annealing decision, if the remaining budget $\rho - \rho_{\text{used}}$ is at most twice the next round's cost, AIM makes the next round the last, and sets its parameters to spend exactly what is left, in the same nine-to-one split:

$$\varepsilon_{t+1} = \sqrt{8 \cdot 0.1\,(\rho - \rho_{\text{used}})}, \qquad \sigma_{t+1} = \sqrt{\frac{1}{2 \cdot 0.9\,(\rho - \rho_{\text{used}})}}. \tag{18}$$

Substituting these into Equation 11 gives 0.1 plus 0.9 of the remainder, which is all of it, so the loop's condition $\rho_{\text{used}} < \rho$ fails after that round and AIM stops. The script's version of both rules is short:

```
def anneal(
    eps: float, sigma: float, change: float, penalty: float, remaining: float
) -> tuple[float, float, bool, bool]:
    """The next round's (eps, sigma). If the last measurement moved the model by less
    than the noise it carried, double eps and halve sigma. Then, if what is left cannot
    pay for two such rounds, make the next round the last one and let it spend exactly
    the remaining budget. Also returns whether it annealed and whether it is the last."""
    annealed = change <= penalty
    if annealed:
        eps, sigma = 2 * eps, sigma / 2
    last = remaining <= 2 * round_cost(eps, sigma)
    if last:
```

```
        eps = math.sqrt(8 * (1 - MEASURE_SHARE) * remaining)
        sigma = math.sqrt(1 / (2 * MEASURE_SHARE * remaining))
    return eps, sigma, annealed, last
```

5.7 The run on the toy dataset

Table 7 lists every round, and Figure 2 plots the workload error of Equation 1 after each. The run is short, because the budget of $\rho = 0.0244$ is small and each annealing quadruples the cost of a round.

Table 7: Every round of AIM on the toy dataset. The model error is the L_1 distance between the true marginal and the model's before the measurement, the noise penalty is $\sqrt{2/\pi}\,\sigma_t$ times the marginal's cells, and the change is how far the model moved on that marginal after refitting. A round anneals when the change is at most the penalty. The workload error is the paper's metric, the weighted L_1 error over the workload divided by the number of marginals and by n.

Round	Marginal chosen	σ_t	ε_t	Cost	Model error on it	Noise penalty	Change	Annealed	Workload error after
1	smoker	38.2	0.017	0.0004	19	61	1	yes	0.1412
2	smoker × condition	19.1	0.035	0.0015	178	61	173		0.1153
3	age × exercise	19.1	0.035	0.0015	249	91	247		0.0761
4	exercise × condition	19.1	0.035	0.0015	79	61	100		0.0722
5	exercise × condition	19.1	0.035	0.0015	22	61	18	yes	0.0726
6	age × condition	9.6	0.070	0.0061	171	46	172		0.0527
7	smoker × condition	7.3	0.091	0.0104	29	23	12	yes	0.0529

Round 1 is the near-random selection discussed above. Measuring smoker again barely moved the model, the change of one record is far below the penalty, so the round annealed. Rounds 2 and 3, at twice the selection budget and half the noise, picked the two marginals with the most

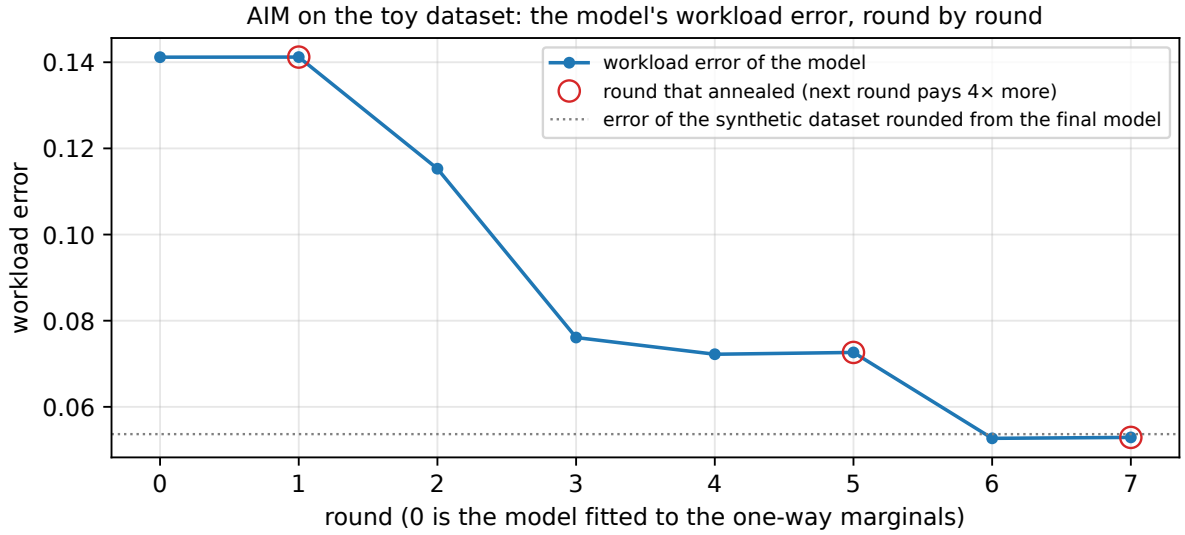


Figure 2: The model's workload error after each round of the toy run. Round 0 is the independence model from the one-way marginals. Circled rounds annealed, so the round after each paid four times as much.

dependence to find, and the workload error fell by almost half. Round 4 picked exercise with condition and fixed it; round 5 picked it again, learned nothing new, and annealed. Round 6, at four times the original stakes, found the dependence between age and condition, the last large error. Round 7 was the final round, set by Equation 18 to spend the rest, and it refined a marginal that was already close. The final error is 0.053, from 0.141 at the start, and Table 8 shows the marginal the note has followed throughout.

Table 8: The marginal over smoker and condition in the toy dataset and in AIM's final model. Compare Table 4, the model before any two-way marginal was measured.

smoker	true: no condition	true: condition	final model: no condition	final model: condition
no	627	98	626.3	103.0
yes	181	94	185.4	104.0

One comparison is owed to the reader. On this toy workload, measuring all six marginals directly with the whole budget, at noise level $\sigma = 11.1$, and fitting the model to them gives a workload error of 0.035, better than AIM's 0.053. Six marginals are few enough for the budget, and none of them is redundant, so there is nothing for selection to gain and the selection budget is spent for nothing. AIM's advantage is on real workloads, where hundreds of marginals compete for the budget and most are nearly implied by a few others. The ablations in Section 8 measure that advantage.

6 Why AIM keeps its promise

Everything AIM does to the data is a Gaussian measurement or an exponential-mechanism selection, and everything else is post-processing. The privacy proof therefore has two parts, which this section sets out: show that each round's cost is Equation 11, and show that the costs never add up to more than ρ. The second part needs care, because the rounds' costs are chosen adaptively, after seeing earlier outputs, and plain composition assumes they are fixed in advance. The tool that handles this is a budget filter.

Lemma 6.1 (Budget filter)**.** *Consider a sequence of mechanisms in which the t-th is ρ_t-zCDP, where ρ_t may be chosen after seeing the outputs of the earlier mechanisms. If, for every choice the earlier outputs can lead to, the sum of the ρ_t over the rounds that run is at most ρ, then releasing all the outputs is ρ-zCDP.*

This is Theorem 3.1 of Feldman and Zrnic (2021), proved for Rényi differential privacy, which contains zCDP as the case of all orders at once. The proof bounds the Rényi divergence of the whole output by a supermartingale argument over the rounds, which is the machinery the plain composition proof does not need.

Theorem 6.1 (AIM is ρ-zCDP)**.** *With the paper's settings, $T = 16d$ and a nine-to-one split, AIM is ρ-zCDP. By Proposition 2.1 (c) it is therefore (ε, δ)-DP for every pair that the conversion gives.*

Proof. The plan is to list every access to the data with its cost, confirm that nothing else touches the data, and then show that the loop's bookkeeping keeps the total at ρ, so that Lemma 6.1 applies.

Step 1: the accesses and their costs. The initialization measures at most d one-way marginals, each at noise level σ_0. By Lemma 2.1 and Proposition 2.2 each costs $1/(2\sigma_0^2)$, so the initialization costs at most $d/(2\sigma_0^2) = 0.9\rho\, d/T$ by Equation 12, which is $0.9\rho/16$ for $T = 16d$. Round t makes two accesses. The selection is the exponential mechanism with the score of Equation 16 at budget ε_t, whose sensitivity Proposition 5.1 gives, so by Proposition 2.3 it costs $\varepsilon_t^2/8$. The measurement is a Gaussian release at level σ_t, which costs $1/(2\sigma_t^2)$. Together they are Equation 11, which is exactly what the algorithm adds to ρ_{used}.

Step 2: nothing else touches the data. The model fits of Equation 9 use only the released measurements. The candidate weights depend on the workload alone, and the candidate filter on the sets already selected and on ρ_{used}, both of which are released. The annealing test of Equation 17 compares two fitted models, and the final-round rule of Equation 18 reads ρ_{used}. The rounding to records uses the final model. By Proposition 2.1 (b), none of this adds to the cost. In particular, the next round's ε_{t+1} and σ_{t+1} are functions of earlier outputs, which is exactly the situation Lemma 6.1 allows.

Step 3: the total never exceeds ρ. After the initialization, by Step 1, the remaining budget is at least $\rho(1 - 0.9/16)$, and the first round costs $\rho/T = \rho/(16d)$ by Equation 12, which is smaller. So the first round fits. Now consider any later round $t+1$, and look at the decision taken at the end of round t. If the final-round rule fired, round $t+1$ costs exactly the remainder, by the arithmetic below Equation 18, so after it $\rho_{\text{used}} = \rho$ and the loop stops. If it did not fire, the

remainder was more than twice the cost of round $t+1$, so that round fits with budget to spare, and the argument repeats at the end of it. Either way ρ_{used} never exceeds ρ, on every path the random choices can take.

By Steps 1 to 3, AIM is a sequence of mechanisms with adaptively chosen costs whose sum is at most ρ on every path, and Lemma 6.1 makes it ρ-zCDP. □

The proof also shows why the final-round rule is not a detail. Without it, a round could begin with less budget left than it costs, and the only remedies would be to overspend or to waste what is left. With it, the last round always spends exactly the remainder, in the same split as every other round. The script checks this at the end of every run, where ρ_{used} equals ρ to floating-point precision.

7 What AIM says about its own error

A synthetic dataset is more useful when it comes with a statement of how far its marginals may be from the truth. AIM provides one, computed from quantities it has already released and so at no extra privacy cost. This section states the simpler of the paper's two bounds and explains where it comes from; the proof is cited.

Consider a marginal r that is contained in one or more of the measured sets. Each such measurement $\tilde{y}_i$ gives, by adding up its cells, a noisy copy of $\mu_r(D)$, and the copies can be averaged, with weights by their precision, into one estimate $\bar{y}_r$. That estimate is the truth plus Gaussian noise of a known level $\bar{\sigma}_r$, computable from the σ_i and the cell counts. The question is how large the L_1 norm of that noise can be.

Theorem 7.1 (A confidence bound for a measured marginal)**.** *Let $\bar{y}_r = \mu_r(D) + Z$, where Z has m_r independent $\mathcal{N}(0, \bar{\sigma}_r^2)$ entries. For every $\lambda \geq 0$, with probability at least $1 - e^{-\lambda^2}$,*

$$\left\|\mu_r(D) - \bar{y}_r\right\|_1 \leq \sqrt{2\ln 2}\,\bar{\sigma}_r\, m_r + \lambda\,\bar{\sigma}_r\sqrt{2m_r}. \tag{19}$$

This is Theorem 3 of the AIM paper. The proof has two steps. The first bounds the expected L_1 norm by Jensen's inequality, which gives the first term and is slightly looser than Lemma 5.1. The second applies a concentration inequality to the L_1 norm, which is a Lipschitz function of the Gaussian vector, and that gives the second term. The bound on the synthetic dataset follows by the triangle inequality: the distance from $\mu_r(\hat{D})$ to the truth is at most its distance to $\bar{y}_r$, which is known, plus the bound above. For a marginal that no measured set contains, the paper gives a weaker bound built from the selection scores. Those are bounded too, because a candidate with a very high score would almost surely have been selected. In the paper's experiments, with λ set for 95% confidence, the bounds held and were within a factor of about four of the true error for measured marginals and eight for the others.

8 How well it works

The paper evaluates AIM on six public datasets with between nine and sixteen attributes, and between about a thousand and a million records. Each is run on three workloads of three-way marginals, at nine privacy budgets from $\varepsilon = 0.01$ to $\varepsilon = 100$, with $\delta = 10^{-9}$. Three of the competitors ignore the workload, among them MST, the winner of a NIST synthetic-data competition. The other three use it, among them MWEM with Private-PGM as its model, and two methods that fit neural networks or relaxed tables instead of a graphical model.

Over all settings, AIM had the lowest workload error in 76% of them, and in every setting with $\varepsilon \geq 1$. Against the next best method in each setting it was better by a factor of 1.6 on average, and by up to 5.7 on the workloads that concentrate on a few attributes, where selecting what to measure matters most.

The more instructive experiment removes AIM's ideas one at a time and measures how much the error grows, as Table 9 reports.

Table 9: The paper's ablation, as geometric means over the datasets and budgets of the ratio of the error without the idea to the error with it.

Idea removed	Error grows by a factor of
Private-PGM, replaced by a relaxed table that need not be a distribution	2.36
adaptive rounds and the annealed budget split, replaced by fixed rounds	1.48
the noise-aware score of Equation 16, replaced by the model error alone	1.37
Gaussian noise, replaced by Laplace noise	1.18
add-or-remove neighbours, replaced by replace-one	1.13
the nine-to-one split, replaced by an even split	1.08
the initialization on one-way marginals	1.03

The two largest factors are the two things this note spent most of its pages on. Generating from a real distribution, which Private-PGM does, matters more than anything else, and the adaptive budget comes second. The two ideas the note presented as AIM's own, the annealing and the noise-aware score, together account for a factor of about two.

Three limits of the method are worth stating. Every attribute must be discrete, so numeric attributes are binned, and the bins are a choice the user makes before any privacy is spent. The model's size is capped, so a workload whose marginals overlap in a tangled way cannot all be measured, however large the budget. And the running time grows with the model, since every round refits it. The paper ran AIM on one processor core with a memory limit of four gigabytes, and the 80 megabyte cap on the model is what keeps that feasible.

9 What the note proves and what it cites

The note proves the facts that are specific to AIM. A marginal has sensitivity one (Lemma 2.1), Gaussian noise has the expected size that sets the score's penalty (Lemma 5.1), the score has the sensitivity the weights give it (Proposition 5.1), and the whole algorithm is private (Theorem 6.1). It cites the general tools. These are the properties of zCDP (Proposition 2.1), the costs of the two building blocks (Proposition 2.2 and Proposition 2.3), the budget filter (Lemma 6.1), the form of Private-PGM's solution (Theorem 4.1) and the error bound (Theorem 7.1). Each citation says what its source proves.

10 Sources

1. R. McKenna, B. Mullins, D. Sheldon and G. Miklau (2022), *AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data*, PVLDB 15(11). The algorithm, its privacy theorem, the error bounds and the experiments. https://arxiv.org/abs/2201.12677
2. R. McKenna, D. Sheldon and G. Miklau (2019), *Graphical-model based estimation and inference for differential privacy*, ICML. Private-PGM: the objective of Equation 9, the form of its solution (Theorem 4.1) and the algorithms that fit it without building the full table. https://arxiv.org/abs/1901.09436
3. M. Bun and T. Steinke (2016), *Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds*. The definition of zCDP, its composition and post-processing, and the Gaussian mechanism's cost. https://arxiv.org/abs/1605.02065
4. C. Canonne, G. Kamath and T. Steinke (2020), *The Discrete Gaussian for Differential Privacy*. The conversion from zCDP to (ε, δ)-DP in Proposition 2.1 (c). https://arxiv.org/abs/2004.00010
5. F. McSherry and K. Talwar (2007), *Mechanism Design via Differential Privacy*, FOCS. The exponential mechanism and its ε-DP guarantee.
6. M. Cesar and R. Rogers (2021), *Bounding, Concentrating, and Truncating: Unifying Privacy Loss Composition for Data Analytics*, ALT. The $\varepsilon^2/8$ zCDP cost of the exponential mechanism. https://arxiv.org/abs/2004.07223
7. V. Feldman and T. Zrnic (2021), *Individual Privacy Accounting via a Rényi Filter*, NeurIPS. The budget filter of Lemma 6.1. https://arxiv.org/abs/2008.11193
8. M. Hardt, K. Ligett and F. McSherry (2012), *A Simple and Practical Algorithm for Differentially Private Data Release*, NeurIPS. MWEM, the ancestor of AIM's select-measure-generate loop. https://arxiv.org/abs/1012.4763